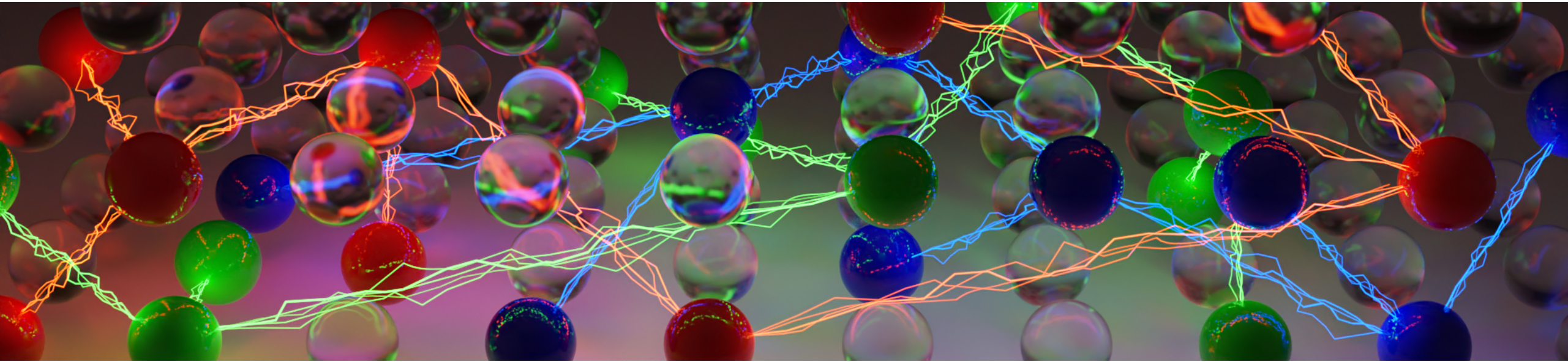




Practical SAT Solving

Lecture 13 – Maximum Satisfiability (MaxSAT)

Markus Iser, Dominik Schreiber | July 28, 2025

Maximum Satisfiability

Today's lecture is based on the slides by Matti Järvisalo presented at 2016 SAT Summer School.



The SAT/SMT/AR Summer School
June 22-25, 2016
Lisbon, Portugal
International Summer School on
Satisfiability, Satisfiability Modulo
Theories, and Automated Reasoning



Deadlines
Application: May 1st
Registration: June 1st

Venue
Instituto Superior Técnico
(IST/UL), University of Lisbon

Organisers
Inês Lynce • Ruzica Piskac •
Philipp Rümmer

Lecturers
Maria Paola Bonacina •
Supratik Chakraborty • Dejan
Jovanović • Matti Järvisalo •
Marius Lindauer • Florian
Lonsing • Nuno P. Lopes •
Jakob Nordström • Andrew
Reynolds • João Marques Silva •
Laurent Simon • Christoph
Weidenbach • Thomas Wies



©2016 SAT Summer School

<http://ssa-school-2016.it.uu.se/programme/#maxSAT>

Maximum Satisfiability (MaxSAT)

Exact Boolean Optimization Paradigm

- Builds on the success story of SAT solving
- Great recent improvements in practical solver technology
- Expanding range of real-world applications

Alternative to Integer Programming

- Solvers provide provably optimal solutions
- Propositional logic as the underlying declarative language:
 - especially suited for inherently “very Boolean” optimization problems

Outline

- Motivation
 - Need for exact optimization
- Basic concepts
 - MaxSAT
 - Complexity
 - Use in practice
- Overview of algorithmic approaches to MaxSAT
 - Branch and bound
 - MaxSAT by integer programming (IP)
 - SAT-based: iterative, core-guided
 - SAT-IP hybrids: Implicit hitting set approach
- Use of SAT solvers for MaxSAT

Optimization

Most real-world problems involve an optimization component.

Examples

- Find a shortest path/plan/execution to a goal state
Planning, model checking, ...
- Find a smallest explanation
Debugging, configuration, ML predictions, ...
- Find a least resource-consuming schedule
Scheduling, logistics, ...
- Find a most probable explanation (MAP)
Probabilistic inference, ...

High demand for automated approaches to finding good solutions to computationally hard optimization problems

Importance of Exact Optimization

Giving Up?

“The problem is NP-hard, so let’s develop heuristics / approximation algorithms”

No! Benefits of provably optimal solutions:

- Resource savings
→ Time, Workforce, Energy, Material, . . .
- Accuracy
- Better approximations
→ by optimally solving simplified problem representations

Key Challenge: Scalability of exactly solving instances of NP-hard optimization problems

Generic Constrained Optimization Paradigms

*Given a conjunction of constraints of the form $\sum_{i=1}^n c_i x_i \leq b$ (with constant coefficients c_i and bound b),
find an assignment to the variables x_i that satisfies all constraints
and that maximizes the objective function $\sum_{i=1}^n c_i x_i$.*

Constrained Optimization Paradigms

- Integer-Linear Programming (ILP)
 - Variables x_i are Integers
 - Algorithms: e.g. Branch-and-Cut with Simplex
- Pseudo-Boolean Optimization (PBO)
 - Variables x_i are Boolean
 - Algorithms: e.g. CDCL-based
- Maximum Satisfiability (MaxSAT)
 - Variables x_i are Boolean, Coefficients $c_i \in \{-1, 1\}$, Bound $b = 1$
 - Algorithms: e.g. CDCL-based

MaxSAT Applications

- Placement/Routing/Debugging/Verification in Hardware Design
- Planning, Scheduling, Resource Allocation
- Product Configuration
- Software Package Management
- Causal Discovery, Argumentation, Formal XAI
- Max-Clique
- ... and many more!

Central to success: Advances in MaxSAT solver technology

MaxSAT: Classic Definition

- **Input:** CNF formula F (set of clauses)
- **Task:** Find an assignment τ that maximizes the number of satisfied clauses

Central Generalizations

- **Weighted MaxSAT:** Each clause C has a weight w_C , and the goal is to maximize the total weight of satisfied clauses.
- **Partial MaxSAT:** Some clauses are hard (infinite weight); soft clauses can be violated.
- **Weighted Partial MaxSAT:** Mix of hard clauses and weighted soft clauses.

Each of these variants can be reencoded such that all soft clauses are unit clauses.

Terminology

- **Solution:** Assignment satisfying all hard clauses
- **Cost:** Sum of weights of falsified soft clauses
- **Optimal Solution:** One that minimizes the cost

Example: Encoding Shortest Paths

- Grid-based shortest path problem from S to G
- Horizontal/vertical moves only; blocked cells not allowed
- Not a practical MaxSAT application, but useful for illustration

n	o		p	q
h	i	j	k	G
c	d	e	l	r
a		f		t
S	b	g	m	u

Example: Encoding Shortest Paths

- Boolean variables: one per unblocked square
- S , G must be visited (hard unit clauses)
- All other squares: soft unit clauses (e.g., $\neg a$) with weight 1 (“Prefer not to visit”)
- MaxSAT minimizes the number of visited squares
- Without further constraints that formulation only visits S and G

n	o		p	q
h	i	j	k	G
c	d	e	l	r
a		f		t
S	b	g	m	u

Example: Encoding Shortest Paths

Ensure a valid path between S and G .

Constraint 1: S and G must have exactly one visited neighbor

- For S : $a + b = 1 \Rightarrow \text{CNF: } (a \vee b), (\neg a \vee \neg b)$
- For G : $k + q + r = 1 \Rightarrow \text{CNF: } (k \vee q \vee r), (\neg k \vee \neg q), (\neg k \vee \neg r), (\neg q \vee \neg r)$

Constraint 2: All other visited squares must have exactly two visited neighbors

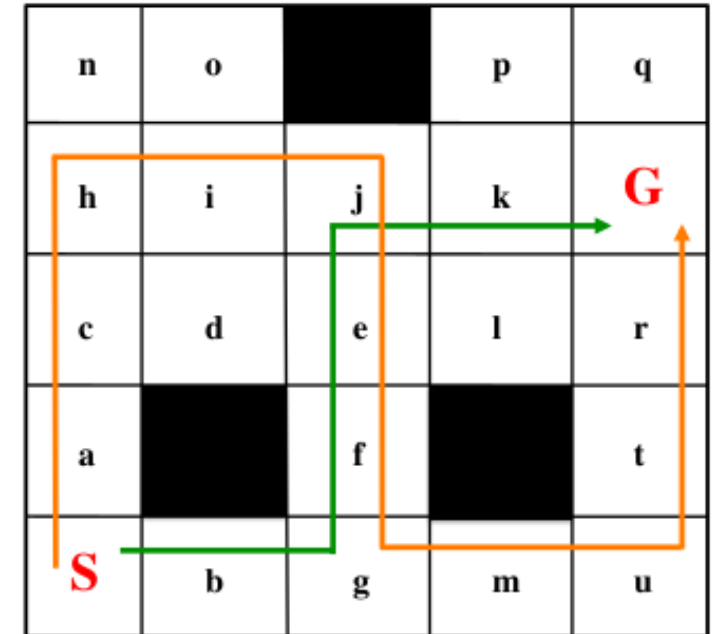
- Example: for square e , if e is visited, then $d + j + l + f = 2$
- Requires encoding a cardinality constraint in CNF
- Efficient cardinality constraint encodings are key for MaxSAT performance, with the typical trade-off between size and complexity

n	o		p	q
h	i	j	k	G
c	d	e	l	r
a		f		t
S	b	g	m	u

Example: Path Properties

Every solution to the hard clauses defines a valid path from S to G .

- Each visited square falsifies a soft clause (e.g., $\neg x$)
- MaxSAT solution is a shortest path (minimum number of visited squares)
 - Orange path: 14 visited squares
 - Green path: 8 visited squares (optimal)



Representing High-Level Soft Constraints

MaxSAT can represent high-level soft constraints compactly.

Based on Cook-Levin Theorem: any $\mathcal{NP}$ constraint $\rightarrow$ polynomial CNF

Softening an $\mathcal{NP}$ -Constraint

- Let $\mathcal{C}$ be a finite-domain soft constraint with weight $W_{\mathcal{C}}$
- Encode $\mathcal{C}$ into CNF: $CNF(\mathcal{C}) = C_1 \wedge C_2 \wedge \dots \wedge C_m$
- Introduce fresh variable a , add hard clauses: $(C_i \vee a)$ for all i
- Add soft clause: $(\neg a)$ with weight $W_{\mathcal{C}}$

MaxSAT: Complexity

- **Decision version:** $\mathcal{NP}$ -complete

- Given CNF F , integer k : is there an assignment satisfying at least k clauses?

- **Optimization version:** $\text{FP}^{\mathcal{NP}}$ -complete

- Solvable with a polynomial number of calls to an $\mathcal{NP}$ oracle
- SAT solver acts as the $\mathcal{NP}$ oracle in practice
- Like TSP: polynomial-time computation using an $\mathcal{NP}$ oracle

- **Hard to approximate:** APX-complete

- Constant-factor approximation possible
- No poly-time approximation scheme (PTAS) unless $\mathcal{P} = \mathcal{NP}$

Practical MaxSAT Solving

Standard Solver Input Format: DIMACS WCNF

- Like DIMACS CNF: Variables indexed from 1 to n , Negation: $-i$ means $\neg x_i$, Clauses terminated with 0
- Header line:
`p wcnf <#vars> <#clauses> <top>`
- Clause weight is first integer in line; if weight $\geq$ top $\rightarrow$ hard clause

Push-Button Solvers / Black-box Solvers

- Input: in standard WCNF format
- Output: provably optimal solution or UNSATISFIABLE
- Internally rely on CDCL SAT solvers to prove unsatisfiability of subsets of clauses
- Examples: Open-source MaxSAT Solvers
 - OpenWBO — <http://sat.inesc-id.pt/open-wbo/>
 - MaxHS — <http://maxhs.org>
 - LMHS — <http://www.cs.helsinki.fi/group/coreo/lmhs/>

Algorithms for MaxSAT Solving

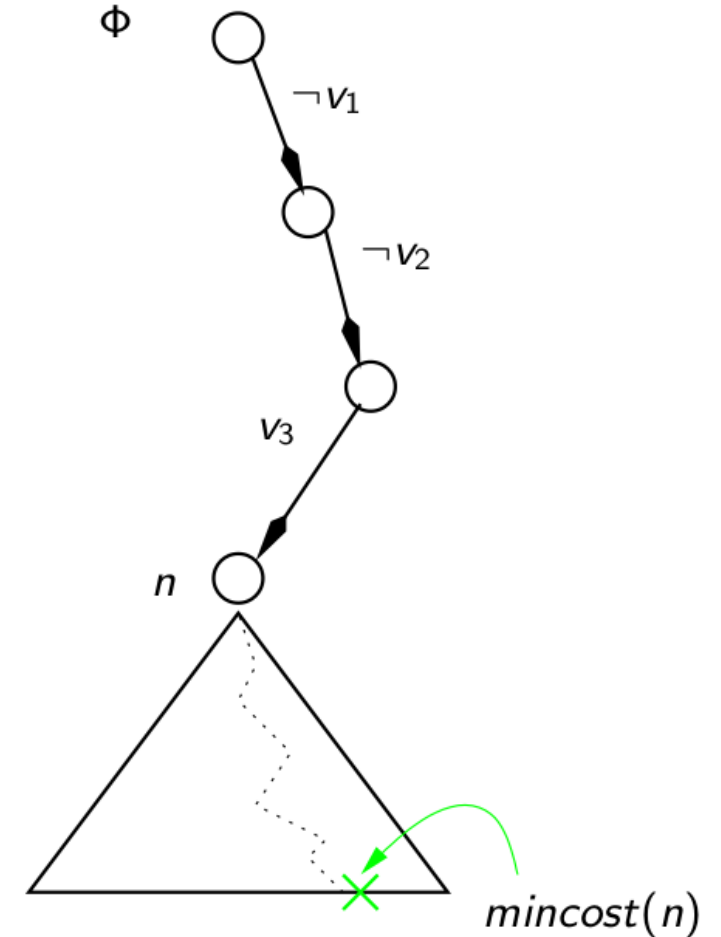
- **Branch and Bound:** MaxSatz, ahmaxsat
- **Direct Integer Programming:** IP Encoding + IP Solver (e.g., CPLEX, Gurobi)
- **Iterative, Model-Based:** QMaxSAT
- **Core-Based:** Eva, MSCG, OpenWBO, WPM, maxino
- **IP-SAT Hybrids:** MaxHS, LMHS

Branch and Bound

Classic method for optimization over search trees

Effective on small, combinatorially hard problems (e.g., Max-Clique), but scalability issues with thousands of variables

- UB = Maintain upper bound (UB) on current best solution cost
- $\text{mincost}(n)$ = minimum cost achievable under node n
- Backtrack if $\text{mincost}(n) \geq \text{UB}$
→ no solution under node n can improve the current best solution UB
- Goal: compute lower bound (LB) such that $\text{mincost}(n) \geq \text{LB}$
- If $\text{LB} \geq \text{UB}$, then backtrack ($\Rightarrow \text{mincost}(n) \geq \text{LB} \geq \text{UB}$)



Branch and Bound: Lower Bounds by Cores

Look for inconsistencies that force some soft clause to be falsified.

- Strategy: find unsatisfiable sets of clauses (cores)
- Each core forces at least one clause to be falsified
- Example:
 - $\kappa = \{(x, 2), (\neg x, 3)\}$ is unsatisfiable; replace with $\kappa' = \{(\emptyset, 2), (\neg x, 1)\}$
 - $\{(x, 2), (\neg x, 3)\} \rightarrow \{(\emptyset, 2), (\neg x, 1)\}$
 - Cost of $\emptyset$ increased by 2
 $\Rightarrow$ 2 is a lower bound
 - The cost of each truth assignment is preserved
- Repeat:
 1. Detect unsatisfiable core κ
 2. Apply sound transformation to increase $\text{cost}(\emptyset)$
 3. Stop if no further LB improvement possible or $\text{LB} \geq \text{UB}$

MaxSAT by Integer Programming (IP)

Using IP solvers as MaxSAT engines.

- IP solvers widely used in Operations Research
- Solvers: IBM CPLEX, Gurobi, SCIP, etc.
- Solve problems with linear constraints and integer variables
- Algorithms: Branch-and-Cut
 - Linear relaxations and cutting planes, i.e., linear constraints that cut off non-integral solutions
 - Sometimes branching on integer variable bounds
 - ... and several other techniques
- Very effective on many standard optimization problems
But do not dominate native MaxSAT solvers on “very Boolean” problems

Relaxing Clauses

- MaxSAT encodings often use **relaxation variables**
- Given soft clause $(x_1 \vee x_2 \vee \dots \vee x_k)$:
 - Add a new variable r to get $(r \vee x_1 \vee \dots \vee x_k)$
 - $r = 1$: clause is relaxed (automatically satisfied)
 - $r = 0$: clause must be satisfied

MaxSAT Encoding into IP

1. Relax each soft clause C_i using a new variable r_i
2. Convert each clause to linear constraint:

$$r_i + x + (1 - y) + z + (1 - w) \geq 1$$

3. Boolean variables become 0-1 bounded integers
4. Objective function:

$$\min \sum_{C_i \in F_s} w_i \cdot r_i$$

SAT-Based MaxSAT Solving

The most widely used modern approach.

- Solve a sequence of SAT instances that ask for different values of k :
Is there a truth assignment falsifying at most k soft clauses?
- SAT-based MaxSAT algorithms mainly do two things:
 1. Develop better ways to encode this decision problem.
 2. Find ways to exploit information obtained from the SAT solver at each stage in the next stage.

Assume unit-weight soft clauses for now . . .

Methods for SAT-Based MaxSAT

- **Iterative Search:** Iteratively increase k until SAT
- **Core-Based Methods:** Use unsatisfiable cores to guide search
- **Hybrid Methods:** Combine SAT solving with integer programming

Iterative Search

Basic Approach

- To check whether F has a solution of cost $\leq k$, solve: $(C_1 \vee r_1) \wedge \cdots \wedge (C_n \vee r_n) \wedge (\sum_{i=1}^n r_i \leq k)$
- Iterate over $k = 1, 2, \dots$ until optimal k is found

Iterating over k

- **Linear Search:** (not efficient)
Start at $k = 1$, increment until SAT
- **Binary Search:** (effective with core-based reasoning)
 - Initialize: $LB = 0$, $UB = \# \text{soft clauses}$
 - Check $k = \lfloor \frac{LB+UB}{2} \rfloor$
 - If SAT: $UB = k$, else $LB = k + 1$
 - Stop when $UB = LB + 1$, then UB is optimal.
- **Linear Search (SAT to UNSAT):** (can be effective)
 - Find model π for hard clauses, let $k = \# \text{violated soft clauses} - 1$
 - Try solving again with lower k until UNSAT
 - If SAT: set k to $\# \text{violated soft clauses}$ and repeat
 - If UNSAT: last SAT solution is optimal

SAT-Based MaxSAT Solving using Cores

Using UNSAT cores to guide solving.

Motivation

Adding linear cardinality constraints over all soft clauses is too loose:

- One relaxation variable r_i per soft clause, could be well over 100k of variables
- Linear cardinality constraints over all soft clauses are too loose:
no information about which relaxation variables to assign to 1
- SAT solver must explore many subsets of soft clauses

Unsatisfiable Cores in MaxSAT

Core-based approach gives more powerful constraint over which particular soft clauses to relax.

- **UNSAT Core:** A subset $F'_s \subseteq F_s$ s.t. $F_h \wedge F'_s$ is UNSAT
- At least one clause in each core must be falsified
- Instead of iteratively ruling out non-optimal solutions, iteratively find and rule out UNSAT cores
- Typically cores are much smaller than full soft clause set

Core-Guided MaxSAT Algorithms: Fu-Malik

- First core-guided MaxSAT algorithm [Fu & Malik, 2006]
- Iterative approach:
 1. Find an UNSAT core
 2. Relax clauses in the core with new variables
 3. Add an AtMost-1 constraint over new relaxation vars
- Repeat until the formula becomes SAT
- Each iteration lowers the cost of solutions by 1 (in the unweighted case)

Fu-Malik: Example

- Initial Formula:

$$\begin{aligned} C_1 &= x_6 \vee x_2, & C_2 &= \neg x_6 \vee x_2, & C_3 &= \neg x_2 \vee x_1, & C_4 &= \neg x_1, & C_5 &= \neg x_6 \vee x_8, \\ C_6 &= x_6 \vee \neg x_8, & C_7 &= x_2 \vee x_4, & C_8 &= \neg x_4 \vee x_5, & C_9 &= x_7 \vee x_5, & C_{10} &= \neg x_7 \vee x_5, \\ C_{11} &= \neg x_5 \vee x_3, & C_{12} &= \neg x_3 \end{aligned}$$

- Core 1: $\{C_3, C_4, C_7, C_8, C_{11}, C_{12}\}$

- Add relax vars r_1 to r_6 , and AMO constraint $\sum r_i \leq 1$

- Core 2: $\{C_1, C_2, C_3, C_4, C_9, C_{10}, C_{11}, C_{12}\}$

- Add relax vars r_7 to r_{14} to these clauses, and AMO constraint $\sum_{i=7}^{14} r_i \leq 1$

- Now the instance is SAT, and the optimal cost is the number of iterations (2 here)

- Final Formula:

$$\begin{aligned} C_1 &= x_6 \vee x_2 \vee r_7, & C_2 &= \neg x_6 \vee x_2 \vee r_8, & C_3 &= \neg x_2 \vee x_1 \vee r_1 \vee r_9, & C_4 &= \neg x_1 \vee r_2 \vee r_{10}, & C_5 &= \neg x_6 \vee x_8 \\ C_6 &= x_6 \vee \neg x_8, & C_7 &= x_2 \vee x_4 \vee r_3, & C_8 &= \neg x_4 \vee x_5 \vee r_4, & C_9 &= x_7 \vee x_5 \vee r_{11}, & C_{10} &= \neg x_7 \vee x_5 \vee r_{12} \\ C_{11} &= \neg x_5 \vee x_3 \vee r_5 \vee r_{13}, & C_{12} &= \neg x_3 \vee r_6 \vee r_{14}, & \sum_{i=1}^6 r_i &\leq 1, & \sum_{i=7}^{14} r_i &\leq 1 \end{aligned}$$

Core-Guided MaxSAT Algorithms: MSU3

MSU3 is a another MaxSAT algorithm for exploiting cores by Marques-Silva and Planes (2007).

Differences to Fu-Malik

- Introduce only at most one relaxation variable to each soft clause
→ Re-use already introduced relaxation variables
- Instead of adding one AtMost-1/Exactly-1 constraint per iteration:
Update the AtMost-k, k noting the k-th iteration
- Relaxed soft clauses become hard

Other Core-Guided Algorithms

- OpenWBO uses MSU3 with incremental cardinality constraints to achieve state-of-the-art performance on many problems. [Martins, Joshi, Manquinho, and Lynce, 2014]
 - Combine with an incremental construction of the cardinality constraint: each new constraint builds on the encoding of the previous constraint
- WPM2 proposes a method for dealing with overlapping cores [Ansótegui, Bonet, and Levy, 2013a]
 - Group intersecting cores into disjoint covers.
The cores might not be disjoint but the covers will be
 - at-most-k constraints over the soft clauses in a cover
 - at-least-k constraint over the clauses in a core

Hybrid MaxSAT Solving: IP + SAT

Combining Integer Programming with SAT solving

Implicit Hitting Set Duality

- SAT-based core-guided MaxSAT $\approx$ hitting set problem
- Cores $\rightarrow$ sets to hit
- Solving hitting set: choose subset of soft clauses to relax
- Solve via integer programming
- SAT solver checks feasibility of choices

MaxHS and LMHS

- **MaxHS** and **LMHS** are IP-SAT hybrid solvers
- Maintain:
 - Set of cores $\Rightarrow$ constraints in hitting set problem
 - IP solver chooses relaxed soft clauses (minimal cost)
 - SAT solver checks whether the set is feasible
- Efficient and robust for many benchmarks

Advantages of IP-SAT Hybrids

- **Scalable:** Many SAT calls, few IP calls
- **Provably optimal:** Solves MaxSAT to optimality
- **Flexible:** Can combine domain-specific knowledge
- **Effective:** Among top performers in MaxSAT Evaluations

Example: MaxHS Pseudocode

1. $R \leftarrow \emptyset$
2. **loop**
 - 2.1 Compute minimum-cost hitting set H for all known cores (via IP)
 - 2.2 Solve SAT on hard clauses F_h and soft clauses $\{C_i : i \notin H\}$
 - 2.3 If SAT: return current assignment
 - 2.4 If UNSAT core κ : add κ to list of cores

Summary

- MaxSAT: Boolean optimization building on SAT
- Practical solvers provide optimal solutions
- Many real-world applications
- Algorithmic approaches:
 - Branch and Bound
 - IP-based
 - SAT-based (Iterative, Core-guided)
 - IP-SAT hybrids (Implicit Hitting Sets)